

DICE-FER: Decoupling Identity Confounders for Enhanced Facial Expression Recognition Using Mutual Information Estimation

Aashika Gupta, Ritika, Krishna Makwana, Akshat Shahjade
EE656 Course Project, Research Paper 3

Abstract—Facial expression recognition (FER) systems face challenges due to variations in individual identity. These identity features act as confounders, reducing the clarity of emotion-specific representations. In this work, we reimplement the DICE-FER (Decoupling Identity Confounders for Enhanced FER) model, which uses mutual information to disentangle identity from expression in facial images. Our report aims to explain the DICE-FER methodology step-by-step and display our understanding of it.

I. INTRODUCTION

Facial expressions are essential to human communication, and automating their recognition through computer vision has numerous applications. However, FER remains difficult due to:

- Small inter-class variation: *Emotions like "neutral" and "slightly happy" may look very similar.*
- Big intra-class variation: *The same emotion can look very different on different people*
- Entanglement: *of identity features (person specifics like lip shape) with expression features (curving of lip) which confuses the model.*

The DICE-FER framework addresses this problem by disentangling expression and identity features using mutual information (MI) estimation in a 2 stage training framework: to extract the expression, and then filter out the identity.

II. RELATED WORK

Traditional approaches to FER often rely on identity-labeled datasets or use GANs to generate synthetic face images that it would compare with original. They use specialized loss functions like island or cluster loss. These methods suffer from label dependency or computational inefficiency. These are the problems DICE-FER was made to avoid.

III. THEORETICAL BACKGROUND

A. Embeddings and Feature Spaces

An embedding is a learned vector representation that maps high-dimensional data (like images) into a lower-dimensional space where similar items are close together. In the context of facial expressions, an embedding vector $E = [e_1, e_2, \dots, e_d]$ might encode abstract features like e_1 = mouth curvature, e_2 = eye squint, etc. Each dimension e_i represents some learned

aspect of the expression, though these features are not human-interpretable.

Now imagine a 512 dimension cartesian space where each axis represents the value of each feature of the vector. This is called the feature space, and the embedding is just a point in it.

Different identities with the same expression would have similar expression embeddings, and hence would cluster close to each other in the feature space.

For FER, once we are able to create accurate expression embeddings, we can train a classifier model on the feature space to predict the answers.

1) *Neural Network Architecture*: Modern FER systems typically use convolutional neural networks (CNNs) as feature extractors for images. A CNN processes an input image tensor of shape $[H, W, C]$ (height, width, channels) through multiple convolutional layers, each applying learned filters to detect patterns at different scales. The final layers produce a fixed-size embedding vector, typically 128-512 dimensions, that encodes the most relevant features for the task.

Once the embeddings are generated, they can be put into a classification model that gives us the final result. This could be K-means, Fuzzy C-means, SVM, neural network, etc.

B. Mutual Information and Statistics

1) *Theoretical MI Formula*:: Mutual Information (MI) measures the amount of shared information between two random variables. MI is calculated as:

$$I(M, Z) = \int \int p(m, z) \log \left(\frac{p(m, z)}{p(m)p(z)} \right) dm dz \quad (1)$$

- M = image (or image patch)
- Z = an embedding
- $p(m, z)$: The probability of seeing the pair (M, Z) together (i.e., the chance that $E(M) = Z$).
- $p(m)$: The probability of seeing image M , regardless of what feature Z it produces.
- $p(z)$: The probability of feature z showing up, no matter which image it came from.

The MI formula compares the actual joint distribution: $p(m, z)$ vs. what we'd get if m and z were independent: $p(m)p(z)$.

If they're very different, knowing m tells you a lot about z (and vice versa) $\Rightarrow$ **High mutual information**

If they're the same, m and z are unrelated $\Rightarrow$ **Zero mutual information**

2) *MINE Approximation:* Because true distribution $p(m, z)$, $p(m)$ and $p(z)$ are not calculatable during training, we use an ANN approximation for the Donsker-Varadhan representation of the theoretical MI formula, proposed in MINE paper:

$$\hat{I}_\theta(M, Z) = \mathbb{E}_{p(m, z)}[T_\theta(m, z)] - \log \mathbb{E}_{p(m)p(z)}[e^{T_\theta(m, z)}] \quad (2)$$

- $\mathbb{E}_{p(m, z)}[T_\theta(m, z)]$ = Average score on real pairs (i.e., image & its own features)
- $\mathbb{E}_{p(m)p(z)}[e^{T_\theta(m, z)}]$ = Average score on randomly mixed pairs (image & unrelated features)
- $T_\theta()$ = statistics network; trained to maximize the above difference

By training to maximize the difference, we reinforce T_θ to give high value when there is more similarity and vice-versa, i.e. training to detect MI.

3) *Global and Local Components of MINE approximation:* MINE actually comprises of calculating both global and local mutual information values, then combining them:

- *Global MI:* Measures information between the entire image and the full embedding vector

$$\hat{I}_{\theta, \text{global}}(M, Z) = \hat{I}_\theta(M, Z) \quad (3)$$

- *Local MI:* Measures information between image patches and the embedding, capturing fine-grained spatial relationships

$$\hat{I}_{\theta, \text{local}}(M, Z) = \sum_{i=1}^{N_{\text{patches}}} I(P^{(i)}(M), Z) \quad (4)$$

– $P^{(i)}(M)$ represents the i^{th} patch of M

In practice, “patches” refers to dividing the image into a regular grid (e.g., 4×4 or 7×7), where each patch is a fixed-size region of the image. This grid division is not adaptive to facial features—it does not use facial landmarks or content-based segmentation. As a result, the same patch index may correspond to different facial regions if the face is not centered or is shifted in the image.

To ensure robustness, it is common to use data augmentation (such as random cropping or translation) or face alignment during training, so that the model learns to handle faces in different positions. Without such measures, performance may drop if the test images are not aligned similarly to the training images. More advanced models may use adaptive or landmark-based patches, but DICE-FER as described uses a simple grid approach.

Incorporating Local MI calculations enhances fine-grained feature detection.

IV. PROPOSED METHOD: DICE-FER

DICE-FER disentangles expression and identity by training on batches of image pairs that share expressions but differ in identity. The model architecture consists of encoders, mutual information estimators, and a discriminator. Training happens in two main stages, with all operations performed on batches for computational efficiency and gradient stability.

A. Forward Pass Stage 1: Learning Expression Features

Input: Batch of image pairs $M_1 N_1, M_2 N_2, \dots, M_B N_B$ where each pair shares the same expression but different identities. Each image is a tensor of shape $[H, W, C]$ where $H = 224, W = 224, C = 3$ (RGB channels), and B is the batch size (typically 16–64).

Output: Batch of expression embeddings with minimal identity bias, trained to capture shared emotional content across all pairs in the batch.

1) *Step 1: Expression Encoding:* Data In: Batch of raw image tensors $M_1, \dots, M_B, N_1, \dots, N_B \in \mathbb{R}^{B \times 224 \times 224 \times 3}$ Convolutional neural network (ResNet-18 backbone) processes entire batch in parallel with final fully connected layer
Data Out: Batch of expression embeddings $E_{M_1}, \dots, E_{M_B}, E_{N_1}, \dots, E_{N_B} \in \mathbb{R}^{B \times 512}$ (512-dimensional vectors)

2) *Step 2: Cross-Image MI Maximization:* Statistics networks T_θ, T_ϕ compute mutual information estimates between an image and its pair's expression embedding, for all pairs in parallel. We maximize this information. We do this to ensure the model learns expression features that are consistent across different people showing the same emotion, by maximizing mutual information between paired images and each other's expression embeddings.

Formula:

$$\mathcal{L}_{\text{expMI}} = \frac{1}{B} \sum_{i=1}^B \left[\mu_{\text{exp}} \left(\hat{I}(M_i, E_{N_i}) + \hat{I}(N_i, E_{M_i}) \right) + \nu_{\text{exp}} \left(\hat{I}(\text{patches}(M_i), E_{N_i}) + \hat{I}(\text{patches}(N_i), E_{M_i}) \right) \right] \quad (5)$$

where $\mu_{\text{exp}}, \nu_{\text{exp}}$ are weighting hyperparameters.

Data Out: Scalar loss value $\mathcal{L}_{\text{expMI}} \in \mathbb{R}$ averaged over the batch

3) *Step 3: Expression Similarity Constraint:* Since M and N have the same expression, $E(M)$ and $E(N)$ must be similar. We take L1 distance averaged over batch

Formula:

$$\mathcal{L}_1 = \frac{1}{B} \sum_{i=1}^B |E_{M_i} - E_{N_i}|_1 = \frac{1}{B} \sum_{i=1}^B \sum_{j=1}^{512} |E_{M_i}^j - E_{N_i}^j| \quad (6)$$

Data Out: Scalar similarity loss $\mathcal{L}_1 \in \mathbb{R}$ averaged over the batch

4) *Step 4: Expression Loss Combination:* We combine two important goals to learn clean expression features:

- $\mathcal{L}_{\text{expMI}}$: encourages the expression features to capture as much emotion-related information as possible by maximizing mutual information.
- $\mathcal{L}_1$: ensures that expression features from two people with the same emotion are similar.

To balance these, we calculate a weighted combination using the hyperparameter δ :

$$\mathcal{L}_{\text{exp}} = \mathcal{L}_{\text{expMI}} - \delta \cdot \mathcal{L}_1$$

We maximize $\mathcal{L}_{\text{exp}}$ so that the model keeps useful emotion information while making expression features consistent across different identities.

B. Forward Pass Stage 2: Learning Identity Features

Input: Same batch of images $M_1, \dots, M_B, N_1, \dots, N_B$ + trained expression embeddings $E_{M_1}, \dots, E_{M_B}, E_{N_1}, \dots, E_{N_B}$

Output: Batch of expression-invariant identity embeddings

1) *Step 1: Identity Encoding:* We use a separate identity encoder (just like the expression encoder) to process each image and get identity features.

$$I_{M_i} = E_{\text{id}}(M_i), I_{N_i} = E_{\text{id}}(N_i) \text{ for } i = 1, \dots, B$$

This helps the model learn what makes each face different from others, apart from emotion.

2) *Step 2: Full Feature Concatenation:* We combine expression and identity features by joining them side by side. $T_{M_i} = [E_{M_i}, I_{M_i}]$, $T_{N_i} = [E_{N_i}, I_{N_i}]$ for $i = 1, \dots, B$
 $T_{M_1}, \dots, T_{M_B}, T_{N_1}, \dots, T_{N_B} \in \mathbb{R}^{B \times 1024} (512 + 512)$
 Now each image has a full feature vector that contains both emotion and identity.

3) *Step 3: Identity MI Maximization:* Data In: Images and their full feature vectors (expression + identity)

What we do: Use a network to check how well the image and its features are connected.

Formula:

$$\mathcal{L}_{\text{idMI}} = \frac{1}{B} \sum_{i=1}^B \left[\mu_{\text{id}} \hat{I}(M_i, T_{M_i}) + \nu_{\text{id}} \hat{I}(\text{patches}(M_i), T_{M_i}) \right] \quad (7)$$

$\mu_{\text{id}}, \nu_{\text{id}}$ are weights, and we average over the batch.

This helps the model learn strong and clear identity features for each image.

4) *Step 4: Adversarial Independence Training:* We add another network called a "discriminator". It tries to guess if the expression (E) and identity (I) came from the same image.

The identity encoder tries to fool this discriminator by keeping identity and expression fully separate — which is what we want.

$D(E, I)$ outputs a number between 0 and 1 (higher = more likely from same image).

$$\mathcal{L}_{\text{adv}} = \frac{1}{B} \sum_{i=1}^B [\log D(E_{M_i}, I_{M_i}) + \log(1 - D(E_{M_i}, I_{N_i}))] \quad (8)$$

We shuffle I_{N_i} to create "fake" pairs.

Over time, the best case is when the discriminator is totally confused and just guesses randomly (outputs 0.5). That means identity and expression are completely separate.

5) *Step 5: Identity Loss Combination:* We now combine both goals — learning good identity features and keeping them separate from expression features:

$$\mathcal{L}_{\text{id}} = \mathcal{L}_{\text{idMI}} - \zeta \cdot \mathcal{L}_{\text{adv}}$$

- ζ : Controls how hard we try to keep identity and expression separate
- $\mathcal{L}_{\text{idMI}}$: Makes sure identity features are meaningful
- $\mathcal{L}_{\text{adv}}$: Makes sure identity doesn't mix with emotion

Data Out: Final identity loss $\mathcal{L}_{\text{id}} \in \mathbb{R}$ for the entire batch

C. Training Process

Total Loss is Computed over the 2 stages for the forward pass, by simple addition.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{exp}} + \mathcal{L}_{\text{id}}$$

Weight Optimization: The model maximizes $\mathcal{L}_{\text{total}}$ with respect to encoder parameters using Adam optimizer with learning rate 10^{-4} . Gradients are computed from the batch-averaged loss, providing stable training signals.

D. Final Classification

After training the disentanglement model, only the expression encoder E_{exp} is retained. A simple classifier (fully connected layer + softmax) is trained on expression embeddings and their labels corresponding.

V. IMPLEMENTATION

A. Framework and Setup

All models were implemented in PyTorch. Data loading and augmentation were handled using `torchvision` and `PIL`. The experiments were run on a local machine with a Ryzen 9 processor, 16GB RAM, and an NVIDIA RTX 3050 GPU (4GB VRAM). Mutual information estimation modules were implemented following the Deep InfoMax and MINE frameworks.

B. Model Architecture

The expression and identity encoders both use a modified ResNet-18 backbone truncated before the final classification layer. Each encoder outputs a 128-dimensional embedding vector.

- **Expression Encoder:** Extracts emotion-relevant features from grayscale face images.
- **Identity Encoder:** Trained to extract identity features that are invariant to expression.
- **Discriminator:** A 3-layer fully connected network trained adversarially to detect entanglement between expression and identity.
- **Statistics Networks:** Shallow MLPs compute global and local mutual information estimates.

C. Data Preprocessing

- Grayscale conversion and resizing to 112×112 .
- Random horizontal flips and $\pm 15^\circ$ random rotations for augmentation.
- Whitening to remove pixel-level redundancy.
- Emotion-labeled image pairs were constructed using the CK+ dataset.

D. Training Strategy

- **Stage 1 (Expression Learning):** The model maximizes mutual information between paired images of the same emotion while minimizing the L1 distance between their embeddings.
- **Stage 2 (Identity Learning):** Identity features are learned by maximizing mutual information while adversarially minimizing overlap with expression features using a discriminator network.
- **Classifier Training:** A shallow classifier is trained on frozen expression embeddings for final FER accuracy evaluation.

E. Training Settings

- **Batch size:** 32
- **Epochs:**
 - Stage 1 (Expression Encoder): 15 epochs
 - Stage 2 (Identity Encoder + Adversarial): 30 epochs
 - Classifier fine-tuning: 10 epochs
- **Optimizer:** Adam
- **Learning Rate:** 1×10^{-3}
- **Loss Weights:**
 - $\lambda_{\text{local}} = 0.001$ (Local MI weight)
 - $\delta = 0.1$ (L1 similarity loss in expression learning)
 - $\zeta_{\text{adv}} = 0.025$ (Orthogonality loss in identity learning)

VI. RESULTS

A. t-SNE Visualization of Learned Features

To visualize the effectiveness of the learned expression and identity embeddings, we applied t-SNE dimensionality reduction.

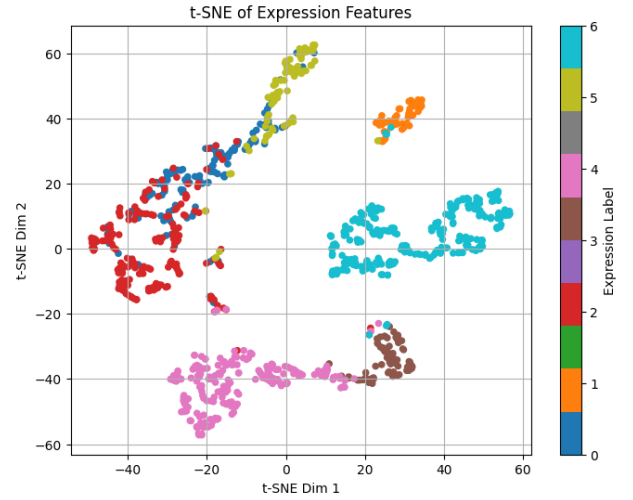


Fig. 1. t-SNE plot of expression embeddings. Each point represents an image, colored by its expression label. Well-separated clusters indicate discriminative expression features.

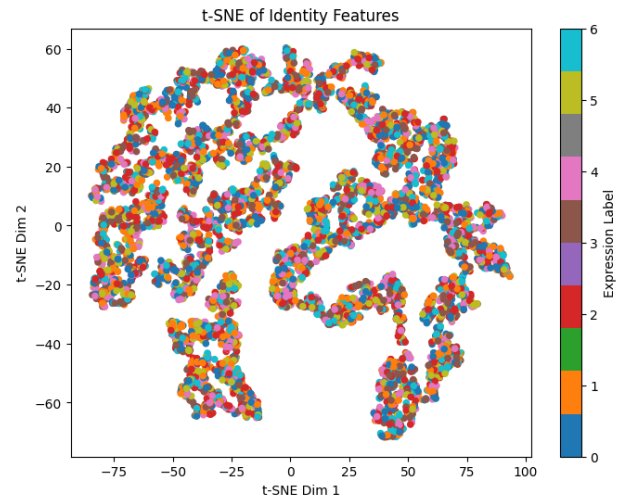


Fig. 2. t-SNE plot of identity embeddings. The uniform layout with no expression-based clustering demonstrates effective disentanglement.

B. Nearest Neighbor Retrieval

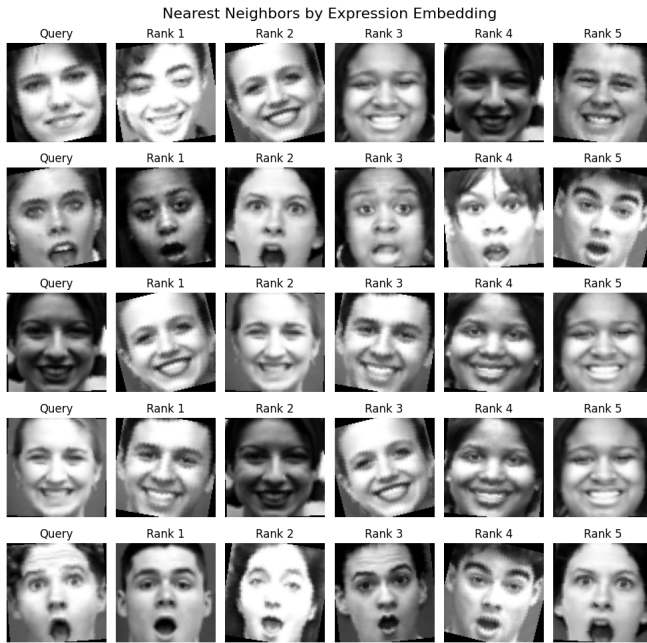


Fig. 3. Top-5 nearest neighbors retrieved using expression embeddings. Neighbors exhibit similar facial expressions, validating semantic structure in the embedding space.

C. Qualitative Expression Clustering



Fig. 4. Qualitative expression clustering. For each query (leftmost), top-5 retrieved images are shown. Consistency in expression and geometry confirms the model’s effectiveness.